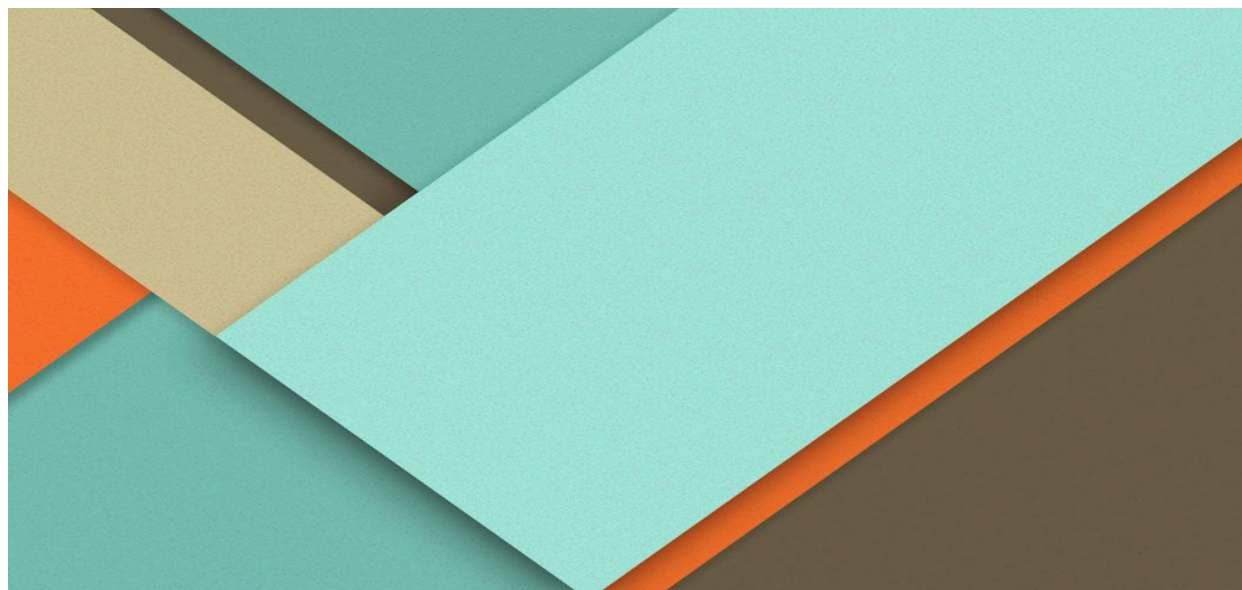




Trabajo Final Obligatorio de Programación Concurrente



Facultad de Informática
UNIVERSIDAD NACIONAL DEL COMAHUE

ANTEZANA de la RIVERA REYES, Alejandro David

FAI - 2945

DOMINIO: PARQUE DE DIVERSIONES

Introducción

El presente proyecto modela el funcionamiento concurrente de un parque de diversiones en Java, coordinando el ciclo de vida de los visitantes, los empleados y el cumplimiento del horario del complejo (apertura 09:00 hs, cierre de ingresos 18:00 hs, cierre de actividades 19:00 hs y parque vacío a las 23:00 hs).

Criterio General del Trabajo

La simulación implementa el modelo de un hilo por entidad activa y objetos pasivos compartidos protegidos mediante encapsulamiento estricto:

- Entidades Activas (Thread / Runnable):
 - Cada Visitante es modelado por un hilo autónomo que recorre el parque de forma no determinista mientras las actividades se encuentren abiertas.
 - RelojParque actúa como temporizador central del complejo, coordinando la apertura (09:00), el fin de ingresos (18:00), el cierre de actividades (19:00) y la finalización del día (23:00).
 - TransporteConCola ejecuta hilos de servicio independientes para el Barco Pirata y el Tren Turístico.
 - EncargadoPremios ejecuta hilos consumidores dedicados al canje en los puestos de juegos.
- Entidades Pasivas (Sectores y Recursos Compartidos):
 - Todas las atracciones implementan la interfaz común ActividadParque (usar, cerrar, getNombre). Esto desacopla a los visitantes de la lógica interna de cada juego, permitiendo un flujo dinámico y polimórfico que facilita modificaciones y permite escalar el sistema. Por ejemplo, con esta implementación, agregar nuevas actividades al parque es sencillo porque solo debemos definir su comportamiento particular.
 - Parque actúa como monitor central para el aforo general y la sincronización del reloj.

Necesidad de Concurrencia	Mecanismo Elegido	Componente
Limitar recursos disponibles que son limitados	Semaphore	Molinetes, Shopping, Capacidad Sala Teatro, Pista Autitos chocadores
Formar grupos exactos y sincronizar partida	CyclicBarrier	Montaña Rusa (5), Autitos (20), Comedor (4), Teatro (5)
Fila bloqueante productor-consumidor con aforo	ArrayBlockingQueue	Barco Pirata, Tren Turístico, Juegos de Premios
Esperar la finalización de un viaje individual	CountDownLatch	SolicitudViaje en Barco y Tren
Intercambio bidireccional atómico entre dos hilos	Exchanger	SolicitudPremio (Ficha - Premio)
Reserva combinada de recursos	ReentrantLock + Condition	Realidad Virtual (Visor, Manoplas, Base)
Coordinación horaria global y vigilar vaciado del predio	Monitor (synchronized, wait, notifyAll)	Parque y Registro

Entrada, Horarios y Salida del Parque

Molinetes: Semaphore

El ingreso posee molinetes limitados modelados mediante:

```
molinetes = new Semaphore(cantidadMolinetes, true);
```

- **Justificación:** Representa recursos homogéneos donde solo importa limitar a lo sumo a k personas simultáneas atravesando la entrada. El parámetro true garantiza atención en orden FIFO, evitando la inanición de hilos en la entrada.
- **Alternativas descartadas:** Una CyclicBarrier obligaría a que siempre lleguen exactamente k visitantes juntos para cruzar; un ReentrantLock limitaría el paso a una única persona a la vez ignorando la existencia de múltiples molinetes.

Apertura, Cierre y Control de Aforo: Monitor

La clase Parque utiliza métodos y bloques synchronized para gestionar de forma atómica: ingresoAbierto, ingresoFinalizado, actividadesAbiertas y personasDentro.

- **Justificación:** Varios hilos observan y modifican estados lógicos compuestos. La guarda en while (!ingresoAbierto && !ingresoFinalizado) { wait(); } protege contra cambios de estado concurrentes, ya que hace que el visitante compruebe que el parque esté abierto antes de continuar.
- **Vaciado seguro:** El método esperarParqueVacio() suspende al reloj en wait() mientras personasDentro > 0. Cada visitante que sale decrementa el contador en un bloque finally y ejecuta notifyAll(), permitiendo al reloj constatar que a las 23:00 no queda nadie dentro del predio.

Montaña Rusa

Esta atracción requiere la combinación de tres responsabilidades coordinadas:

1. lugarDeEspera (Semaphore): Limita la sala de espera. El visitante ejecuta tryAcquire(); si está lleno, no se bloquea, retorna false y continúa hacia otra actividad del parque.
2. asientos (Semaphore(5)): Reserva los 5 lugares físicos del carro. Ni bien se adquiere el asiento, se libera inmediatamente el permiso de lugarDeEspera en su respectivo bloque finally, permitiendo que nuevos visitantes entren a la fila mientras el carro anterior viaja.
3. barreraSalida (CyclicBarrier(5)): Garantiza que el carro parta únicamente cuando los 5 asientos estén ocupados.

- Protocolo de Cierre: El método cerrar() desactiva la atracción (abierta = false) e invoca barreraSalida.reset(), destrabando de inmediato a cualquier pasajero atrapado en turnos incompletos mediante BrokenBarrierException para que desaloje la atracción.

Autitos Chocadores

Con 10 autos de 2 personas cada uno, el turno exige exactamente 20 visitantes para iniciar:

- Sincronización: Se utiliza un Semaphore(20, true) para reservar la pista y una CyclicBarrier(20) que retiene a los participantes hasta que los 10 autos estén completos.
- Justificación: El semáforo protege la pista impidiendo el solapamiento del turno siguiente durante el recorrido; la barrera sincroniza el arranque simultáneo.
- Cierre: Al llegar las 19:00 hs, cerrar() ejecuta barrera.reset(), cancelando turnos incompletos y liberando a los hilos.

Barco Pirata y Tren Turístico

Ambas atracciones extienden la clase TransporteConCola bajo el patrón de productor–consumidor:

- Cola de Espera (ArrayBlockingQueue): Ordena las solicitudes (SolicitudViaje) con capacidad fija. Si la fila se llena, offer() retorna false inmediatamente sin bloquear al visitante.
- Partida Dual (Capacidad vs. Tiempo): El hilo de servicio consume el primer pasajero y ejecuta poll(restante, TimeUnit.MILLISECONDS). El viaje parte cuando se alcanza la capacidad total o cuando expira el tiempo límite (esperaMaxima), lo que ocurra primero.
- Sincronización de Pasajeros (CountDownLatch): Cada SolicitudViaje posee su propio CountDownLatch(1). Cuando el conductor termina el recorrido, invoca terminar() (countDown()) sobre el lote seleccionado, liberando exclusivamente a los pasajeros de ese viaje.

Área de Juegos de Premios

Modela el intercambio atómico de fichas por premios entre visitantes y encargados:

- Cola de Puesto (ArrayBlockingQueue): Distribuye las solicitudes entre los encargados disponibles en orden FIFO.
- Intercambio Atómico (Exchanger): Cada SolicitudPremio encapsula un Exchanger privado. Se realizan dos encuentros sincronizados (*rendezvous*):
 1. El visitante entrega su Ficha con el puntaje obtenido y el encargado la recibe.
 2. El encargado evalúa el puntaje (pequeño, mediano, grande) y entrega el objeto Premio de vuelta al visitante.
- Finalización: En el cierre, se introducen objetos centinela con identificador "FIN", garantizando que los encargados bloqueados en take() despierten y finalicen su ejecución de forma ordenada.

Comedor

- Sincronización: Utiliza un Semaphore(mesas * 4, true) para el aforo total del salón y una CyclicBarrier(4) para que los 4 comensales de una mesa comiencen a comer estrictamente al mismo tiempo.
- Rechazo no bloqueante: El comensal ejecuta capacidad.tryAcquire(); si no hay sillas libres, no genera cola artificial y se retira a otra actividad.
- Cierre: Al invocarse cerrar(), mesa.reset() destraba mesas a medio llenar mediante BrokenBarrierException.

Espectáculo en el Teatro

- Sincronización: Combina un Semaphore(20, true) para la capacidad total de la sala y una CyclicBarrier(5) para hacer cumplir la restricción de que el ingreso se realice en grupos completos de 5 personas.
- Justificación: Permite que 4 tandas sucesivas de 5 ingresen a la sala sin exceder el cupo máximo de 20 espectadores.

Realidad Virtual

Requiere adquirir simultáneamente un conjunto de recursos de diferente tipo: 1 visor, 2 manoplas y 1 base.

- Sincronización: Se protegen los contadores bajo un `ReentrantLock(true)` con una `Condition` `equipoDisponible`.
- Política Todo o Nada: El visitante evalúa `while (abierta && (visores < 1 || manoplas < 2 || bases < 1)) { equipoDisponible.await(); }`. Si falta algún elemento, no retiene ninguno, eliminando la condición de "retener y esperar" (*hold and wait*).
- Devolución: Se reponen los 4 componentes dentro de la misma sección crítica y se ejecuta `signalAll()` para despertar a todos los hilos en espera.
- Cierre: `cerrar()` adquiere el lock, marca `abierta = false` y dispara `signalAll()`, liberando a los hilos en espera que retornan `false` y abandonan el sector.

Shopping y Registro Centralizado

Shopping Modela un área comercial y de descanso con capacidad concurrente limitada (N plazas). A diferencia de los juegos mecánicos, el acceso y permanencia son individuales e independientes, por lo que no requiere sincronización por lotes ni barreras grupales.

- Justificación: Se utiliza un `Semaphore(capacidad, true)` para gestionar permisos en orden estricto de llegada (FIFO), evitando la inanición de los visitantes en espera. La adquisición y devolución del permiso se protegen mediante bloques `try-finally` para asegurar que el aforo no sufra fugas ante interrupciones.

Registro Centralizado de Eventos (`Registro.informar`) Centraliza en una clase el seguimiento de todos los hilos en ejecución mediante una salida formateada por consola.

- Justificación: Al ser un método `public static synchronized`, utiliza el monitor de la clase `Registro` para serializar las operaciones sobre `System.out`. Esto garantiza la atomicidad al calcular el tiempo transcurrido (`System.currentTimeMillis()` - INICIO), obtener el nombre del hilo invocador (`Thread.currentThread().getName()`) e imprimir el mensaje, evitando que las líneas de texto se intercalen.